

SIMATS ENGINEERING

ASSESSMENT TOOL 01

COURSE NAME: Ethical Hacking

COURSE CODE: ITA1410

COURSE OUTCOME COVERED

CO4: Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.

QUESTIONS: 01

Total Marks:100

Annexure A

SECURITY TESTING SIMULATION TASK

Code of Conduct:

I K Nitheesh Kumar 192325090 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Nitheesh Kumar

S.No.	Assessment Criterion	Excellent	Good	Satisfactory	Needs Improvement	Marks
1	Tool Selection and Configuration(20)	Selects appropriate open-source tools, configures them correctly, and clearly explains the purpose of each tool.	Selects suitable tools and configures them with minor errors or limited explanation.	Uses basic tools with partial configuration and limited understanding.	Selects inappropriate tools or fails to configure them correctly.	
2	Testing Procedure and Execution(20)	Follows a systematic, authorized, safe, and well-documented testing procedure within the approved scope.	Performs the major testing steps correctly with minor omissions.	Completes only basic testing steps with limited documentation.	Testing procedure is incomplete, unsafe, unauthorized, or poorly executed.	
3	Identification and Validation of Vulnerabilities(20)	Accurately identifies vulnerabilities, manually validates findings, and distinguishes confirmed issues from false positives.	Identifies and validates most important findings with minor gaps.	Identifies basic vulnerabilities but provides limited validation.	Findings are inaccurate, unsupported, or based only on automated tool output.	
4	Risk Analysis and Remediation(20)	Clearly explains the cause, likelihood, impact, severity, and practical remediation for every confirmed vulnerability.	Provides suitable risk analysis and recommendations with minor omissions.	Provides general risk descriptions and basic remediation measures.	Risk ratings are unsupported or recommendations are impractical or unrelated.	
5	Evidence, Report Quality, and Ethical Compliance(20)	Provides clear commands, screenshots, outputs, tables, references, and a professional report while maintaining authorization, confidentiality, and ethical conduct.	Provides sufficient evidence and a well-organized report with minor presentation issues.	Provides limited evidence or an adequately organized report.	Evidence is missing, the report is unclear, or ethical and legal requirements are ignored.	
					Total	

Annexure A

NOTE:

- Any testing performed outside the faculty-approved laboratory target, authorized scope, or permitted time must be treated as a serious violation. Destructive testing, denial-of-service activities, unauthorized password attacks, data modification, and testing of public or third-party systems are strictly prohibited.
- Use any Suitable Open Source Tool

Question 1: Web Server Reconnaissance and Vulnerability Analysis

A cybersecurity analyst is assigned an authorized and isolated vulnerable web server for a security assessment. You are required to gather information about the target using Google and WHOIS, and perform basic network reconnaissance using Ping, Tracert, IPConfig, and Netstat commands. Use Nmap to identify open ports, running services, and service versions. Use WhatWeb to identify the web technologies and frameworks used by the server. Examine HTTP response headers using Curl/Wget and identify information disclosure. Perform CGI and web-server vulnerability scanning using Nikto, and use Gobuster/Dirb to discover hidden directories and files. Analyze the identified administrative pages, backup files, configuration files, and vulnerable resources. Finally, classify the findings based on severity and impact, and recommend appropriate web-server hardening and information-disclosure prevention measures. Submit the commands executed, screenshots, observations, findings, and recommendations.

Question 2: Web Application Vulnerability Assessment

A cybersecurity analyst is assigned an authorized vulnerable web application in an isolated laboratory environment for security testing. You are required to identify the application's technologies and attack surface using appropriate reconnaissance techniques, examine input fields, parameters, cookies, sessions, and HTTP requests, and use suitable tools to identify vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), insecure authentication, session-management weaknesses, directory traversal, and security misconfigurations. Use tools such as Burp Suite, OWASP ZAP, Nikto, Nmap, Curl, and browser developer tools to analyze the application. Test the identified vulnerabilities only within the authorized laboratory environment, document the evidence and potential impact of each finding, classify the vulnerabilities according to severity, and recommend appropriate remediation and secure coding practices. Submit the commands/tool configurations, screenshots, observations, vulnerability findings, severity ratings, impacts, and recommendations.

Question 3: Security Header and HTTPS Configuration Assessment

A cybersecurity analyst is assigned an authorized web application hosted in an isolated laboratory environment to assess its HTTPS and security-header configuration. You are required to use Curl to examine HTTP response headers and Nmap SSL scripts to analyze the server's HTTPS/TLS configuration. Identify missing or improperly configured security headers such as Content-Security-Policy (CSP), HSTS, X-Frame-Options, and

X-Content-Type-Options, and verify whether HTTP requests are automatically redirected to HTTPS. Examine the TLS configuration for weak protocols, insecure cipher suites, and certificate-related issues. Analyze the security purpose and potential impact of each misconfiguration, classify the findings according to their severity, and recommend appropriate HTTPS, TLS, certificate, and security-header hardening measures. Submit the commands used, screenshots, observations, identified risks, severity classification, and remediation recommendations.

Question 4: Broken Access-Control Assessment

A cybersecurity analyst is assigned an authorized student portal in an isolated laboratory environment containing separate student and administrator test accounts. Using OWASP ZAP, capture and analyze requests generated while accessing student records and identify test object identifiers in URLs, forms, cookies, or API requests. Using only the test identifiers provided by the faculty, determine whether one student account can access another student's records or whether a normal user can directly access administrator-only resources. Compare the server responses for authorized and unauthorized requests, explain the difference between authentication and authorization failures, classify the identified access-control weaknesses based on their severity and impact, and recommend appropriate server-side authorization checks, role validation, and secure object-reference mechanisms. Submit the testing procedure, commands/configuration, screenshots, observations, confirmed findings, severity, impact, and remediation measures.

Question 5: Automated Scanning and Manual Validation

A cybersecurity analyst is assigned an authorized web application hosted in a controlled cybersecurity laboratory for vulnerability assessment. Use Nikto to identify web-server misconfigurations and outdated components, and use Wapiti or OWASP ZAP to perform an automated web-application security scan. Compare the vulnerabilities reported by the selected tools and categorize them into misconfiguration, information disclosure, authentication, and injection issues. Select at least three findings for safe manual validation within the authorized laboratory environment and determine whether each finding is confirmed, a false positive, or requires further investigation. Assign an appropriate severity level based on likelihood and potential impact, and recommend practical remediation measures for all confirmed vulnerabilities. Submit a comparative report containing tool outputs, screenshots, validation evidence, severity classification, remediation measures, and overall conclusions.

CO4 – ASSESSMENT TOOL 01

Answered Security Testing Simulation Report

Course: Ethical Hacking | Course Code: ITA1410

Course Outcome Covered: CO4 – Implement network security testing methods by performing web server attacks, analyzing web application vulnerabilities, and assessing wireless network security using Kali Linux.

Scope and ethics: All activities in this report are intended only for the isolated, faculty-approved vulnerable laboratory target. The assessment prohibits testing outside the approved scope, destructive testing, denial-of-service activity, unauthorized password attacks, data modification, and testing of public or third-party systems. 穀filecite學turn0file0學L116-L120像

Question 1: Web Server Reconnaissance and Vulnerability Analysis

1.1 Objective

The objective is to gather information about the authorized web server, identify exposed ports and services, fingerprint web technologies, inspect HTTP response headers, discover accessible resources, and analyze web-server weaknesses. The workflow should progress from passive discovery to controlled scanning and then manual validation.

1.2 Tool Selection and Purpose

Tool	Purpose
Google	Passive discovery of indexed information relevant to the approved target.
WHOIS	Review domain registration and administrative information where applicable.
Ping	Check network reachability and response behavior.
Tracert / traceroute	Observe the route toward the authorized target.
IPConfig / ip addr	Review local network configuration.
Netstat / ss	Review local connections and listening sockets.
Nmap	Identify open ports, services and versions.
WhatWeb	Identify web technologies and frameworks.
Curl / Wget	Inspect HTTP responses, headers and redirects.
Nikto	Scan for common web-server weaknesses and risky resources.
Gobuster / Dirb	Discover directories and files.

1.3 Commands and Procedure

```
whois <authorized-domain>

ping -c 4 <authorized-ip>

traceroute <authorized-ip>

ip addr

ss -tulpen

nmap -sV <authorized-ip>

whatweb http://<authorized-target>
```

```

curl -I http://<authorized-target>
curl -i -L http://<authorized-target>
nikto -h http://<authorized-target>
gobuster dir -u http://<authorized-target> -w <approved-wordlist>
dirb http://<authorized-target> <approved-wordlist>

```

1.4 Observations

- Record the target IP and all open ports identified by Nmap.
- Record service names and versions and compare them with the expected lab configuration.
- Record technologies identified by WhatWeb.
- Inspect Server, X-Powered-By, Location and other relevant response headers.
- Manually verify discovered directories and files before reporting them as findings.
- Preserve sanitized command output and screenshots as evidence.
- Do not treat scanner output alone as proof of a vulnerability.

1.5 Findings and Remediation

Finding	Severity	Impact	Remediation
Unnecessary server/version disclosure	Low	Supports technology fingerprinting.	Minimize unnecessary Server/X-Powered-By disclosure.
Exposed backup/configuration file	High	May expose source code, credentials or settings.	Remove it, restrict access and rotate exposed secrets.
Unprotected administrative page	Medium–High	Expands attack surface.	Enforce authentication, authorization and network restrictions.
Outdated vulnerable service	High/Critical if confirmed	May enable compromise.	Patch/upgrade and disable unnecessary services.

1.6 Web-Server Hardening

- Close unused ports and disable unnecessary services.
- Patch the operating system, web server and frameworks.
- Restrict administrative interfaces to authorized users.
- Remove backups and configuration files from web roots.
- Apply least-privilege permissions.
- Reduce verbose errors and unnecessary technology disclosure.
- Enable logging and monitoring.

Question 2: Web Application Vulnerability Assessment

2.1 Objective

The assessment examines the application's technologies and attack surface, including input fields, parameters, cookies, sessions and HTTP requests. The specified tools can identify SQL injection, XSS, authentication weaknesses, session-management weaknesses, directory traversal and security misconfiguration.

2.2 Procedure

- Map login, search, profile, record and administrative functions available in the lab.
- Inspect request methods, parameters, cookies, response codes and redirects.
- Review cookie attributes such as Secure, HttpOnly and SameSite.
- Capture representative requests with Burp Suite or OWASP ZAP.
- Use browser developer tools to inspect network behavior.
- Use only faculty-provided accounts and test data.

2.3 Representative Commands

```
nmap -sV <authorized-ip>

nikto -h http://<authorized-target>

curl -i http://<authorized-target>/login

curl -i -X POST http://<authorized-target>/login

zaproxy
```

2.4 Vulnerability Categories

Category	Validation Focus	Potential Impact
SQL Injection	Determine whether database input is safely parameterized.	Unauthorized database access/manipulation.
XSS	Check whether untrusted output is safely encoded.	Script execution in another user's browser.
Authentication	Assess login behavior using test accounts.	Unauthorized account access.
Session Management	Inspect rotation, logout and cookie security.	Session theft/fixation/persistence.
Directory Traversal	Check controlled path handling.	Unauthorized file disclosure.
Security Misconfiguration	Inspect debug output, defaults and settings.	Information disclosure and increased attack surface.

2.5 Secure Coding Recommendations

- Use parameterized queries or safe ORM mechanisms.
- Validate input and contextually encode untrusted output.
- Use secure password hashing.
- Rotate sessions after authentication and invalidate them at logout.
- Set Secure, HttpOnly and suitable SameSite cookie attributes.

- Perform authorization checks server-side on every protected endpoint.
- Disable debug mode and remove development artifacts.

Question 3: Security Header and HTTPS Configuration Assessment

3.1 Objective

The assessment evaluates HTTP security headers, HTTP-to-HTTPS redirection, TLS protocols, cipher suites and certificate configuration on the authorized application.

3.2 Commands

```
curl -I http://<authorized-target>
curl -I https://<authorized-target>
curl -i -L http://<authorized-target>
nmap --script ssl-enum-ciphers -p 443 <authorized-ip>
nmap --script ssl-cert -p 443 <authorized-ip>
```

3.3 Security Headers

Header	Purpose	Risk if Missing
Content-Security-Policy	Restricts content and script sources.	Greater XSS/content-injection exposure.
Strict-Transport-Security (HSTS)	Instructs browsers to use HTTPS.	Greater downgrade/insecure-HTTP exposure.
X-Frame-Options	Controls framing of pages.	Greater clickjacking exposure.
X-Content-Type-Options	Reduces MIME-type sniffing.	Unexpected content interpretation.

3.4 TLS and Certificate Remediation

- Disable obsolete or weak TLS protocol support.
- Remove weak/legacy cipher suites.
- Verify certificate hostname, validity and chain.
- Redirect HTTP to HTTPS.
- Avoid mixed content.
- Repeat the checks after remediation.

Question 4: Broken Access-Control Assessment

4.1 Objective

The objective is to determine whether student and administrator test accounts receive only resources permitted by their roles. The assessment requires comparison of authorized and unauthorized requests using only faculty-provided test identifiers.

4.2 Testing Procedure

- Log in with the student test account and capture an authorized request.

- Identify the permitted test object identifier.
- Use another faculty-provided identifier only where explicitly allowed.
- Compare status codes, response bodies and server behavior.
- Request an administrator-only resource with the student account inside the lab.
- Record whether the server denies unauthorized access server-side.

```
curl -i -b '<student-session-cookie>'
http://<authorized-target>/student/<test-id>

curl -i -b '<student-session-cookie>'
http://<authorized-target>/admin/<approved-test-resource>
```

4.3 Authentication vs Authorization

Concept	Meaning	Failure Example
Authentication	Verifies who the user is.	Invalid user is accepted as logged in.
Authorization	Determines what an authenticated user may access.	Student retrieves another student's protected record.

4.4 Risk and Remediation

A confirmed cross-user or cross-role access-control weakness can be High or Critical depending on the sensitivity of the exposed information and actions available. The primary control is server-side authorization for every protected object and endpoint.

- Validate identity, object ownership and role on the server.
- Do not rely on client-side checks or hidden UI elements.
- Use consistent role-based access control.
- Use indirect object references where appropriate, but never as a substitute for authorization.
- Return safe denial responses without leaking protected data.
- Log repeated authorization failures.

Question 5: Automated Scanning and Manual Validation

5.1 Objective

Automated scanning provides broad coverage but can produce false positives. Therefore, automated results must be compared and at least three findings should be manually validated safely in the authorized laboratory.

5.2 Automated Scanning

```
nikto -h http://<authorized-target> -output nikto_report.txt

zapproxy # run the approved scan configuration in the lab
```

Keep scanner outputs as evidence and avoid aggressive or destructive options unless explicitly permitted.

5.3 Comparison

Category	Automated Evidence	Manual Validation	Status
Misconfiguration	Tool reports weak/default setting.	Inspect actual response/configuration.	Confirmed / False Positive / Further Investigation
Information Disclosure	Tool reports banner/resource.	Verify with Curl/browser.	Confirmed / False Positive / Further Investigation
Authentication	Tool reports possible weakness.	Compare behavior with test accounts.	Confirmed / False Positive / Further Investigation
Injection	Tool reports possible input issue.	Controlled non-destructive lab validation.	Confirmed / False Positive / Further Investigation

5.4 Three Manual Validation Examples

Finding A – Information Disclosure

If Nikto or ZAP reports a verbose server banner, inspect the exact response using Curl. If unnecessary software/version information is actually returned, the finding is confirmed and is generally Low severity unless the disclosure materially increases exploitation risk.

```
curl -I https://<authorized-target>
```

Finding B – Missing Security Header

If a scanner reports a missing security header, inspect the HTTPS response and confirm that the header is consistently absent on the relevant endpoint before assigning severity.

```
curl -s -D - https://<authorized-target>/ -o /dev/null
```

Finding C – Access-Control Weakness

If a scanner indicates a possible access-control issue, use only permitted test identifiers and accounts. A response containing another user's protected test record confirms the issue; a correct server-side denial may indicate a false positive or require further investigation.

```
curl -i -b '<student-session-cookie>'  
https://<authorized-target>/resource/<test-id>
```

5.5 Severity Matrix

Severity	Interpretation	Example
Critical	Very high impact with realistic major compromise potential.	Broad remote compromise or highly sensitive data exposure.
High	Significant confidentiality, integrity or availability impact.	Confirmed cross-user sensitive-data access.
Medium	Meaningful but constrained impact.	Moderate authorization weakness.
Low	Limited impact, often defense-in-depth.	Minor version disclosure.
Informational	Observation without material security impact by itself.	Benign technology identification.

5.6 Evidence and Reporting

For every confirmed finding, include the title, affected component, tool, sanitized command/configuration, screenshot or output evidence, observation, manual validation result, severity, likelihood, impact, root cause where supported, remediation and retest result. Redact credentials, session cookies, personal information and other secrets from screenshots.

Overall Conclusion

The five tasks form a complete security-testing workflow. Reconnaissance establishes the attack surface; web-application assessment examines input, session and configuration weaknesses; HTTPS/header assessment checks transport and browser protections; access-control testing verifies server-side authorization; and automated scanning supplies broad coverage that must be followed by manual validation. A high-quality submission distinguishes observed evidence from assumptions, reports validated vulnerabilities, justifies severity using likelihood and impact, and provides practical remediation and retesting steps.

Appendix A: Evidence Checklist

- Authorization and approved target scope recorded.
- WHOIS/passive discovery evidence captured where applicable.
- Ping, traceroute, IP configuration and socket evidence recorded.
- Nmap service/version output saved.
- WhatWeb fingerprint recorded.
- Curl/Wget header evidence saved.
- Nikto output saved.
- Gobuster/Dirb output saved.
- Burp Suite or OWASP ZAP request/response evidence saved.
- TLS/certificate evidence saved.
- At least three automated findings manually validated.
- Severity, impact and remediation documented.
- Sensitive information redacted from screenshots.

Appendix B: Finding Register

ID	Finding	Component	Severity	Validation	Status	Remediation
F-01	Technology/version disclosure	HTTP response	Low	Curl	Complete from lab evidence	Minimize unnecessary headers
F-02	Exposed backup/configuration resource	Web root	High	Curl/browser	Complete from lab evidence	Remove/restrict sensitive files
F-03	Missing security header	HTTPS response	Low/Medium	Curl	Complete from lab evidence	Configure appropriate header
F-04	Broken access control	Test endpoint/API	High	ZAP/Curl	Complete from lab evidence	Server-side object/role checks
F-05	Possible injection issue	Application input	Medium/High if confirmed	Manual lab test	Complete from lab evidence	Parameterized queries and validation